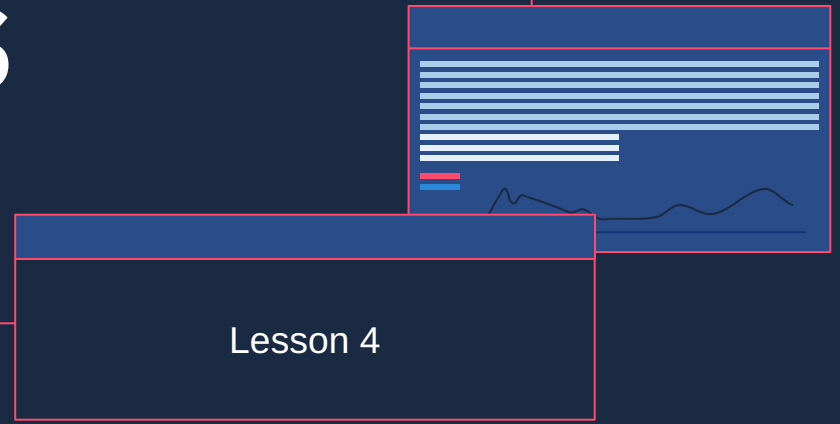
GIRLS WHO CODE: LISTS



Lesson 4

OVERVIEW OF LESSON

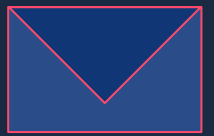
01 ICE BREAKER

02 WOMEN IN TECH

03 LESSON

04 CODING PRACTICE

05 REFLECTION + DISCUSSION



Ice Breaker: Word Chains

We'll start with a category of your choosing.
Someone will come up with a related word.
Then, the next person in your group will come up with a word that starts with the LAST LETTER of the word that the person before them said and so on.

If you take more than 20 seconds to answer, you are eliminated from the game until the next round!

Example: (Category Pokemon)
Person 1: jigglypuff
Person 2: flareon
Person 3: natu



Women in Tech: Karlie Kloss

While watching, think about:

- **Share something you learned about Karlie Kloss!** (What is Kode with Klossy?)
- **Why do you think it's important that we bring more girls in tech?**
- **People who code have a diverse range of interests and backgrounds. What else does Karlie do besides code? What are your non-coding hobbies and interests?**



Review: Conditional Statements!

Conditional statements (if/elif/else) control which code runs depending on whether a condition is true or false.

`if:`	Runs only if condition is true. Example: if 3+2 >= 4: print(“5 >= 4 is true!”)

`elif:` (“else if”)	Runs only if above condition is false and this condition is true. Example:
	if 3 > 4:
	print(“3 > 4 is true!”)
	elif: “hello” == “hello”:
	print(“hello is true!”)

`else:`	Runs if all above conditions are false and this condition is true. Example:
	if 3 > 4:
	print(“3 > 4 is true!”)
	elif: “hello” == “goodbye”:
	print(“hello is true!”)
	else:
	print(“nothing is true!”)

Picture this!

You need to go to the grocery store and you have a bunch of items you need to get. You look in your fridge one by one, how do you remember everything you need?



Lists

A list is a built in **data structure** in Python.

Lists are **ordered, changeable,** and **allow duplicate values.**

- When you add new items to a list, they are automatically placed at the end

Lists can also have multiple data types. You can hold both a string and an int in the same list!

List Example

Let’s say we need to make our grocery list..

```
grocery_list = [] #represents an empty list
```








```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”] #initiate list with existing values
```

Other types of lists

As mentioned before, lists can include different data types.



```
dice_faces = [1,2,3,4,5,6,7,8] #ints

true_or_false = [True, False] #bool

piplup = [“Piplup”, 16, 11.5, “Water”, True]
#profile representing name, height, weight,
type, and if can evolve
```



Accessing values

What if we want to know what the first item in our grocery list is?

In Python, lists are 0-indexed, meaning the first item in a list is in the **0th position**

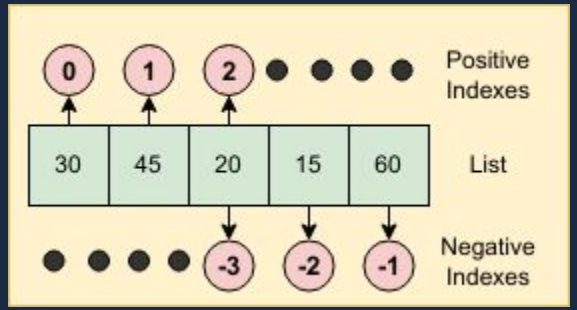
- **var_name = list_name[index]**

```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”]
grocery_item = grocery_list[0] #returns “apple”
```

Accessing values

What about the last item in the list? We could just do grocery_list[4], but what if we don't know how long the list is?

Python also **supports negative indexing!** Typing -1 for the index moves the pointer **to the end of the list.**



Accessing values

Oh no, we only have enough money for the first three items! Let’s make a new list only including the first three items.

- **var_name = list_name[start:stop]** #start is included in list, stop is NOT

```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”]
new_grocery_list = grocery_list[0:3] #returns [‘apple’, ‘eggs’, ‘carrots’]
```

Accessing values

What if we want to pull **every OTHER** item in our list instead of the first three?

- var_name = list_name[start:stop:**step**]
  #how many positions to jump forward

```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”]
new_grocery_list = grocery_list[0::2] #returns [‘apple’, ‘carrots’, ‘ice cream’]
```

Appending values

You're craving mac and cheese tonight, but you don't have macaroni noodles… Instead of making a whole new grocery list, you add to the end of the existing list with **.append**!

- **list_name.append(item)**

```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”]
grocery_list.append(“macaroni”)
#grocery_list = ['apple', 'eggs', 'carrots', 'steak', 'ice cream', 'macaroni']
```

Inserting values

Oh no, grandma is all out of oatmeal :(Let's need to prioritize the oatmeal by inserting it FIRST in the list using **.insert**!

- **list_name.insert(index, item)**

```
grocery_list = [“apple”, “eggs”, “carrots”, “steak”, “ice cream”, “macaroni”]
grocery_list.insert(0, “oatmeal”)
# ['oatmeal', 'apple', 'eggs', 'carrots', 'steak', 'ice cream', 'macaroni']
```

Replacing values

You don’t really want carrots anymore, and now you really want broccoli. Let’s replace “carrots” with “broccoli”

- **list_name[index] = “new_value”**

```
grocery_list = [“oatmeal”, “apple”, “eggs”, “carrots”, “steak”, “ice cream”, “macaroni”]
grocery_list[3] = “broccoli”
# ['oatmeal', 'apple', 'eggs', 'broccoli', 'steak', 'ice cream', 'macaroni']
```

Removing values

Now, you do not want to buy eggs anymore, and you'll postpone getting eggs to next grocery trip. We can remove the first occurrence of eggs using **.remove**!

- **list_name.remove(existing_value)**

```
grocery_list = [“oatmeal”, “apple”, “eggs”, “carrots”, “steak”, “ice cream”, “macaroni”]
grocery_list.remove(“eggs”)
# ['oatmeal', 'apple', 'broccoli', 'steak', 'ice cream', 'macaroni']
```

Deleting values

What if we want to remove an item at a specific index of our list? Or just delete our list completely?

- **del list_name[index] and del list_name**

```
grocery_list = ['oatmeal', 'apple', 'broccoli', 'steak', 'ice cream', 'macaroni']
```

del grocery_list[3] removes ‘steak’

del grocery_list deletes our **WHOLE** list

Removing all items

What if you want to make a new grocery list? We can clear our whole list easily using **.clear**!

- **list_name.clear()**

```
grocery_list = [“oatmeal”, “apple”, “eggs”, “carrots”, “steak”, “ice cream”, “macaroni”]
grocery_list.clear()
# []
# list is now empty!
```

Additional Methods

Length	len(list_name)
Max and Min	max(list_name) and min(list_name)
Combine list	[list1] + [list2] = [new list with all elements of list 1 and list 2 combined]
Extend list	list1.extend(list2) adds contents from list2 TO LIST1

Reflection!






That’s it for this week!

